

Téma 1: Algoritmy, abstraktní datové typy, principy a implementace, shrnutí

(vytvořeno s podporou nástroje ChatGPT, <https://chat.openai.com/>)

Algoritmus je explicitně definovaný formální postup, kterým lze řešit určitý problém nebo vykonat určitou úlohu. Jedná se o exaktní sekvenci kroků nebo instrukcí, které specifikují, jakým způsobem dosáhnout požadovaného výsledku z daného vstupu. Mezi základní vlastnosti algoritmů patří zejména:

- **Přesnost:** algoritmus musí být jednoznačně definován a jeho jednotlivé kroky musí být jasně a srozumitelně popsány. Musí být přesně stanoveno, jaké operace se mají provést a v jakém pořadí.
- **Determinismus:** pro každý vstupní stav a vstupní data musí mít jednoznačně definovaný výstup. Každý krok algoritmu musí být definován tak, aby nemohlo docházet k jeho náhodnému nebo nepředvídatelnému chování.
- **Konečnost:** činnost algoritmu musí být vždy ukončena po konečném počtu kroků a musí poskytnout výsledek. Nemůže se zacyklit nebo nekonečně pokračovat bez výstupu.
- **Efektivita, výkonnost:** dobrý algoritmus by měl vyžadovat co nejméně času a prostoru pro provádění a dosahování výsledků. K měření a hodnocení tohoto ukazatele se používá analýza složitosti.
- **Obecnost, tj. použitelnost** pro různé vstupy, konfigurace a problémy, příslušející do jeho třídy. Neměl by se tedy omezovat pouze na specifické scénáře.
- **Opakovatelnost:** algoritmus by měl dávat stejný výsledek pro stejné vstupy, bez ohledu na čas nebo místo provedení. To umožňuje jeho opakování a ověření správnosti.

Datové struktury jsou kontejnery nebo formáty používané k efektivní organizaci a uložení dat, umožňující efektivní operace a snadný přístup. Poskytují způsob, jak reprezentovat a manipulovat s daty v různých formách, v závislosti na požadavcích různých algoritmů a aplikací. Mezi jejich typické zástupce patří např. pole (array), spojový seznam (linked list), zásobník (stack), fronta (queue), strom (tree), graf (graph), hashovací tabulka (hash table) nebo halda (heap).

Abstraktní datové typy (ADT) jsou logické, matematické či jinak přesně formalizované modely, které definují sadu operací a chování kolekce dat, aniž by specifikovaly podrobnosti jejich implementace. Poskytují způsob, jak abstraktně popsat chování datových struktur, zaměřující se na to, jaké operace lze provádět a jaké vlastnosti vykazují. Mezi jejich typické zástupce patří např. zásobník (stack), fronta (queue), seznam (list), množina (set), mapa (map) nebo prioritní fronta (priority queue).

Datové struktury lze chápat jako konkrétní implementace ADT. Poskytují specifické implementační detaily potřebné k realizaci abstraktních chování definovaných ADT. Pro stejný ADT lze použít více datových struktur. Například zásobník lze implementovat pomocí pole nebo spojového seznamu. Volba konkrétní datové struktury závisí na specifických požadavcích řešeného problému, včetně jeho majoritních operací, efektivitě nebo paměťových či jiných omezení.

STL (Standard Template Library) je klíčovou součástí jazyka C++ a poskytuje programátorům mocný nástroj pro práci s datovými strukturami a algoritmy. Její modulární architektura a bohatá funkčnost usnadňují vývoj robustních a efektivních programů. Je navržena s důrazem na efektivitu a opakovanou použitelnost (reuse). Je postavena na konceptu obecných šablon, které umožňují vytvářet generický kód nezávislý na konkrétních datových typech. Poskytuje optimalizované implementace a algoritmy, které jsou připravené k použití bez nutnosti opakovaného psaní kódu.

Pro práci s daty poskytuje STL různé typy **kontejnerů**:

- Vektor (vector): Dynamicky se rozšiřující pole, umožňující rychlý přístup k prvkům.
- List: Jednosměrně nebo obousměrně vázaný seznam prvků.
- Fronta (queue): Fronta prvků s FIFO (First-In, First-Out) principem.
- Zásobník (stack): Zásobník prvků s LIFO (Last-In, First-Out) principem.
- Množina (set): Kolekce unikátních prvků, seřazených podle hodnoty.
- Mapa (map): Kolekce dvojic klíč-hodnota, kde každý klíč je jedinečný.

Knihovna má dále zabudované obecné **algoritmy**, umožňující provádět operace na datech v kontejnerech a to v následujících oblastech:

- Řazení (sorting): Algoritmy pro řazení prvků v kontejneru, například quicksort, mergesort.
- Hledání (searching): Algoritmy pro vyhledávání prvků v kontejneru, například binary search.
- Transformace (transforming): Algoritmy pro transformaci prvků v kontejneru, například mapování, filtraci.
- Akumulace (accumulating): Algoritmy pro akumulaci hodnot v kontejneru, například součet, průměr.

Jako obecný způsob pro přístup k prvkům v kontejnerech využívá STL **iterátory** (obdoba ukazatelů), které poskytují rozhraní pro procházení a manipulaci s daty v kontejnerech.